

Ringkasan

Materi Pertemuan 1 ini merupakan fondasi awal dalam mempelajari *Sistem Basis Data Relasional*. Pada pertemuan ini, mahasiswa diperkenalkan pada konsep dasar sistem basis data relasional, model data relasional, serta arsitektur *Database Management System* (DBMS) yang menjadi landasan bagi seluruh topik pada pertemuan berikutnya. Materi disampaikan melalui metode ceramah dan diskusi, serta dievaluasi menggunakan quiz.

Sesuai dengan Sub-CPMK-1.1, tujuan pembelajaran pada pertemuan ini adalah agar mahasiswa mampu *menjelaskan* (level kognitif C2) konsep dasar sistem basis data relasional, model data relasional, dan arsitektur DBMS pada Oracle maupun PostgreSQL.

Topik-topik utama yang akan dibahas mencakup:

- Konsep dasar basis data: definisi data, informasi, basis data, dan sistem basis data serta perbedaannya dengan pemrosesan berbasis berkas (file-based).
- Sistem basis data relasional: sejarah singkat, karakteristik, keunggulan, serta peran DBMS dalam pengelolaan data organisasi.
- Model data relasional: konsep relasi (tabel), atribut, tuple, domain, key (primary key, foreign key), dan integritas relasional.
- Arsitektur DBMS: arsitektur tiga skema (three-schema architecture), independensi data, serta komponen utama DBMS.
- Arsitektur Oracle dan PostgreSQL: gambaran umum komponen instance, storage, dan proses pada kedua DBMS sebagai perbandingan praktis.

Melalui materi ini, mahasiswa diharapkan memperoleh pemahaman konseptual yang kuat sebagai bekal untuk mempelajari desain skema, normalisasi, SQL, serta optimisasi performa pada pertemuan selanjutnya. Pemahaman awal ini penting untuk membangun kemampuan menganalisis kebutuhan dan merancang solusi basis data yang efektif dan relevan.

Pendahuluan dan Tujuan Pembelajaran

Selamat datang pada pertemuan pertama mata kuliah *Sistem Basis Data Relasional*. Pertemuan ini merupakan fondasi paling mendasar yang akan menopang seluruh perjalanan pembelajaran Anda selama satu semester. Sebelum kita mampu merancang skema basis data yang optimal, menulis kueri SQL yang efisien, melakukan normalisasi data, atau mengelola transaksi yang aman, kita harus terlebih dahulu memahami konsep-konsep fundamental yang menjadi tulang punggung teknologi basis data itu sendiri. Ibarat membangun sebuah gedung pencakar langit, kita tidak dapat langsung memasang jendela dan atap tanpa memahami bagaimana pondasi, kolom, dan struktur penopangnya bekerja. Pertemuan ini adalah pondasi tersebut.

Dalam kehidupan sehari-hari, kita senantiasa berinteraksi dengan sistem basis data, sering kali tanpa kita sadari. Ketika Anda memeriksa surel di pagi hari, membeli barang di toko swalayan dan kasir memindai barang tersebut, melakukan transaksi perbankan melalui aplikasi ponsel, atau bahkan sekadar mencari informasi di mesin pencari, di balik semua aktivitas tersebut terdapat sistem basis data yang bekerja secara diam-diam namun sangat andal. Sebagaimana dijelaskan dalam materi ajar Oracle, bahkan situs sebesar Yahoo.com menggunakan basis data Oracle untuk menyimpan datanya. Pemahaman tentang bagaimana data disimpan, dikelola, dan diakses adalah kompetensi yang sangat esensial bagi setiap profesional teknologi informasi.

Sesuai dengan capaian pembelajaran yang tertuang dalam Sub-CPMK-1.1, tujuan utama pertemuan ini adalah agar mahasiswa mampu *menjelaskan* konsep dasar sistem basis data relasional, model data relasional, serta arsitektur *Database Management System* (DBMS) yang diterapkan pada Oracle maupun PostgreSQL. Kata kerja "menjelaskan" berada pada level kognitif C2 (Memahami) dalam Taksonomi Bloom. Ini berarti Anda tidak hanya diminta menghafal definisi, tetapi lebih penting lagi, mampu menginterpretasikan konsep, membandingkan berbagai

pendekatan, memberikan contoh ilustratif, dan menjelaskan hubungan antarkonsep dengan bahasa Anda sendiri.

Secara lebih terperinci, setelah menyelesaikan pertemuan ini, Anda diharapkan mampu mencapai beberapa tujuan pembelajaran berikut:

- Membedakan secara jelas antara konsep data, informasi, basis data, dan sistem basis data, serta menjelaskan hubungan di antara keempatnya.
- Menjelaskan kelemahan pendekatan pemrosesan berbasis berkas (*file-based system*) dan bagaimana sistem basis data mengatasi kelemahan tersebut.
- Menguraikan sejarah singkat, karakteristik, dan keunggulan sistem basis data relasional serta peran strategis DBMS dalam pengelolaan data organisasi.
- Menjelaskan komponen-komponen model data relasional seperti relasi, atribut, tuple, domain, serta konsep kunci (*primary key* dan *foreign key*) dan integritas relasional.
- Menggambarkan arsitektur tiga skema (*three-schema architecture*) dan konsep independensi data.
- Membandingkan gambaran umum arsitektur DBMS Oracle dan PostgreSQL dari sisi *instance*, penyimpanan, dan proses.

Metode pembelajaran yang digunakan pada pertemuan ini adalah ceramah dan diskusi, sementara asesmen dilakukan melalui quiz. Oleh karena itu, sangat penting bagi Anda untuk menyimak setiap konsep dengan saksama, mencatat istilah-istilah kunci, dan aktif bertanya maupun berdiskusi. Pemahaman yang kuat pada pertemuan ini akan sangat menentukan kelancaran Anda dalam mengikuti materi-materi lanjutan seperti desain skema, normalisasi, bahasa SQL, manajemen transaksi, hingga optimisasi performa. Mari kita mulai perjalanan intelektual ini dengan penuh keingintahuan.

Latar Belakang dan Konteks

Historis Teknologi Basis Data

Untuk memahami mengapa sistem basis data relasional menjadi begitu dominan dalam dunia teknologi informasi saat ini, kita perlu menempatkannya dalam konteks

historis yang lebih luas. Pada masa-masa awal komputasi bisnis, sekitar dekade 1960-an dan 1970-an, organisasi mengelola data mereka menggunakan sistem berkas tradisional (*traditional file system*). Dalam pendekatan ini, setiap aplikasi atau departemen memiliki kumpulan berkasnya sendiri yang terpisah. Misalnya, departemen keuangan memiliki berkas data karyawan sendiri, departemen sumber daya manusia memiliki berkas data karyawan yang lain, dan departemen produksi juga menyimpan datanya secara terpisah. Pendekatan ini tampak sederhana pada awalnya, tetapi menyimpan banyak masalah serius.

1960s Sistem Berkas Tradisional 1968 Model Hierarkis 1969 Model Jaringan 1970 Model Relasional oleh Codd Kini RDBMS Modern Oracle PostgreSQL Evolusi Teknologi Basis Data

Lini masa perkembangan teknologi basis data dari sistem berkas hingga model relasional modern

Sebagaimana diuraikan dalam buku *Introduction to Database Management System* karya Satinder Bal Gupta dan Aditya Mittal, sistem berkas tradisional memiliki sejumlah kelemahan mendasar. Kelemahan pertama adalah redundansi data (*data redundancy*), yaitu situasi di mana data yang sama disimpan berulang kali di berbagai tempat. Ketika data karyawan disimpan di departemen keuangan dan departemen sumber daya manusia secara terpisah, terjadi pemborosan ruang penyimpanan dan, yang lebih berbahaya, muncul potensi inkonsistensi data (*data inconsistency*). Bayangkan jika seorang karyawan pindah alamat; jika alamat hanya diperbarui di satu berkas dan tidak di berkas lainnya, maka organisasi akan memiliki dua versi kebenaran yang saling bertentangan.

Kelemahan lain dari sistem berkas tradisional mencakup kesulitan dalam mengakses data secara fleksibel, isolasi data yang tersebar dalam berbagai format berkas, masalah integritas data yang sulit ditegakkan, kesulitan dalam menangani akses secara bersamaan oleh banyak pengguna (*concurrency*), serta lemahnya aspek keamanan data. Selain itu, terdapat masalah ketergantungan program

terhadap data (*program-data dependence*), di mana perubahan pada struktur penyimpanan data mengharuskan modifikasi pada seluruh program aplikasi yang mengaksesnya. Kondisi ini membuat pengembangan dan pemeliharaan sistem menjadi sangat mahal dan rentan kesalahan.

Menyadari berbagai keterbatasan tersebut, para peneliti mulai mengembangkan konsep yang lebih terstruktur. Model data pertama yang muncul adalah model hierarkis (*hierarchical model*) yang mengorganisasi data dalam struktur pohon, kemudian disusul oleh model jaringan (*network model*) yang menggunakan struktur graf. Kedua model ini merupakan kemajuan besar dibandingkan sistem berkas, tetapi masih memiliki keterbatasan berupa kompleksitas navigasi dan kekakuan struktur. Pengguna harus memahami jalur fisik penyimpanan data untuk dapat mengaksesnya.

Titik balik terbesar dalam sejarah teknologi basis data terjadi pada tahun 1970, ketika seorang ilmuwan komputer bernama Dr. Edgar Frank Codd yang bekerja di IBM mempublikasikan makalah berjudul "A Relational Model of Data for Large Shared Data Banks". Dalam makalah ini, Codd memperkenalkan model relasional, sebuah pendekatan revolusioner yang mengorganisasi data dalam bentuk tabel (relasi) yang didasarkan pada teori matematika himpunan dan logika predikat. Gagasan Codd sangat elegan karena memisahkan cara data disimpan secara fisik dari cara pengguna memandang dan mengakses data secara logis. Inilah cikal bakal seluruh DBMS relasional modern seperti Oracle, PostgreSQL, MySQL, dan SQL Server yang kita kenal saat ini.

Dalam konteks mata kuliah ini, pemahaman terhadap latar belakang tersebut sangat penting. Kita akan menggunakan DBMS Oracle dan PostgreSQL sebagai contoh implementasi praktis. Oracle, melalui platform Oracle APEX, akan menjadi salah satu lingkungan pembelajaran SQL kita, sementara PostgreSQL sebagai DBMS bersumber terbuka (*open source*) yang sangat populer akan menjadi pembanding yang kaya. Dengan memahami akar historis dan permasalahan yang

melatarbelakangi kemunculan basis data relasional, Anda akan lebih menghargai mengapa konsep-konsep yang akan kita pelajari dirancang sedemikian rupa.

Konsep Dasar Basis Data: Data, Informasi, dan Sistem Basis Data

Untuk membangun pemahaman yang kokoh, kita harus memulai dari terminologi paling fundamental. Banyak orang menggunakan istilah "data" dan "informasi" secara bergantian dalam percakapan sehari-hari, padahal dalam disiplin ilmu basis data, keduanya memiliki makna yang berbeda dan berjenjang. Memahami perbedaan ini adalah langkah pertama menuju penguasaan konsep basis data.



Ilustrasi konseptual transformasi data mentah yang tersebar menjadi informasi yang terstruktur dan bermakna melalui proses pengolahan.

Data dan Informasi

Data adalah fakta-fakta mentah (*raw facts*) yang belum diolah dan belum memiliki makna tertentu bagi penggunaannya. Data dapat berupa angka, huruf, simbol, teks, gambar, suara, atau kombinasi darinya. Sebagai contoh, angka "85", nama "Budi Santoso", atau tanggal "2026-09-27" adalah data. Berdiri sendiri, potongan-potongan data ini tidak banyak berarti. Kita tidak tahu apakah "85" adalah nilai ujian, berat

badan, atau suhu ruangan. Data hanyalah bahan baku yang menunggu untuk diproses.



Diagram alur transformasi data menjadi informasi melalui pemrosesan dan konteks

Informasi, di sisi lain, adalah data yang telah diproses, diorganisasi, atau diberi konteks sehingga menjadi bermakna dan bermanfaat bagi pengambilan keputusan. Ketika kita menyatakan bahwa "Budi Santoso memperoleh nilai 85 pada mata kuliah Sistem Basis Data pada tanggal 27 September 2026", maka data-data mentah tadi telah berubah menjadi informasi yang bermakna. Informasi memberikan jawaban atas pertanyaan seperti "siapa", "apa", "kapan", dan "di mana". Hubungan antara data dan informasi dapat dinyatakan secara konseptual sebagai proses transformasi berikut:

$$\text{Informasi} = f(\text{Data}, \text{Konteks}, \text{Pemrosesan})$$

$$\text{Informasi} = f(\text{Data}, \text{Konteks}, \text{Pemrosesan})$$

Selain data dan informasi, terdapat pula konsep **metadata**, yaitu "data tentang data". Metadata menjelaskan karakteristik data lain, misalnya tipe data suatu kolom, panjang maksimum, apakah kolom boleh kosong, dan sebagainya. Sebagai contoh, keterangan bahwa kolom "nilai" bertipe numerik dengan rentang nilai 0 hingga 100 adalah metadata. Metadata biasanya disimpan dalam sebuah kamus data (*data dictionary*), yang merupakan repositori terpusat berisi definisi seluruh elemen data dalam sebuah basis data.

Basis Data dan Sistem Basis Data

Basis data (*database*) adalah kumpulan data yang saling berhubungan secara logis, disimpan secara terorganisasi dan sistematis, sehingga dapat diakses dan dikelola dengan mudah oleh berbagai pengguna dan aplikasi. Kata kunci di sini adalah "saling berhubungan secara logis" dan "terorganisasi". Basis data bukan sekadar tumpukan data acak, melainkan koleksi data yang dirancang dengan tujuan tertentu, merepresentasikan aspek-aspek dari dunia nyata yang disebut sebagai *miniworld* atau *universe of discourse*. Contohnya adalah basis data akademik sebuah universitas yang merepresentasikan mahasiswa, dosen, mata kuliah, dan nilai.

Database Management System (DBMS) adalah perangkat lunak sistem yang memungkinkan pengguna untuk membuat, mendefinisikan, mengelola, memelihara, dan mengontrol akses terhadap basis data. DBMS bertindak sebagai perantara antara pengguna/aplikasi dengan data fisik yang tersimpan di media penyimpanan. Contoh DBMS yang populer meliputi Oracle Database, PostgreSQL, MySQL, Microsoft SQL Server, dan IBM Db2. Sementara itu, sistem basis data (*database system*) adalah istilah yang lebih luas yang mencakup keseluruhan lingkungan, yaitu gabungan dari perangkat keras (*hardware*), perangkat lunak DBMS, data itu sendiri, prosedur, serta pengguna.

Perlu dibedakan pula antara perangkat lunak sistem (*system software*) dan perangkat lunak aplikasi (*application software*). Berdasarkan penjelasan dalam materi Oracle APEX, perangkat lunak sistem terdiri atas program tingkat rendah yang dirancang untuk berinteraksi langsung dengan perangkat keras komputer, seperti sistem operasi, kompilator, dan utilitas sistem. Sebaliknya, perangkat lunak aplikasi mencakup program-program yang digunakan pengguna akhir seperti pengolah kata, permainan, surel, grafik, dan basis data. DBMS dapat dikategorikan sebagai perangkat lunak sistem yang mengelola sumber daya data, sementara aplikasi seperti Oracle APEX dibangun di atasnya untuk memudahkan pengguna.

Materi Oracle memberikan analogi yang sangat bagus: menggunakan sebuah aplikasi ibarat mengemudikan mobil. Untuk mengemudi, Anda cukup tahu cara

mengoperasikannya tanpa harus memahami secara mendetail bagaimana transmisi bekerja atau bagaimana bahan bakar diubah menjadi tenaga. Aplikasi memiliki "cangkang" ramah pengguna yang menyembunyikan kerumitan di baliknya. Namun, sebagai calon profesional basis data, tugas kita justru mempelajari "mesin" di balik cangkang tersebut, agar mampu merancang dan mengoptimalkan sistem, bukan sekadar menggunakannya.

Perbandingan Sistem Berkas dengan Sistem Basis Data

Perbedaan mendasar antara pemrosesan berbasis berkas dan sistem basis data dapat diringkas dalam beberapa aspek. Pada sistem berkas tradisional, setiap aplikasi mengelola berkasnya sendiri, menyebabkan redundansi tinggi, inkonsistensi, dan ketergantungan program terhadap data. Pada sistem basis data, data dikelola secara terpusat oleh DBMS, sehingga redundansi dapat dikontrol, konsistensi dijaga, integritas ditegakkan, keamanan diterapkan, dan akses bersamaan oleh banyak pengguna dapat dikoordinasikan dengan aman. Keunggulan-keunggulan inilah yang menjadikan sistem basis data sebagai standar de facto dalam pengelolaan data organisasi modern.



Perbandingan sistem berkas terpisah dengan sistem basis data terpusat melalui DBMS

Sistem Basis Data Relasional dan Peran DBMS

Setelah memahami konsep dasar, kini kita masuk ke inti mata kuliah ini, yaitu sistem basis data relasional. Sistem basis data relasional adalah sistem basis data yang menerapkan model data relasional yang diperkenalkan oleh Dr. E. F. Codd. DBMS yang mengimplementasikan model ini disebut *Relational Database Management System* (RDBMS). Oracle dan PostgreSQL adalah dua contoh RDBMS terkemuka yang akan kita gunakan sepanjang perkuliahan ini.

Karakteristik Sistem Basis Data Relasional

Karakteristik utama yang membedakan sistem relasional dari model lainnya adalah bahwa seluruh data disajikan kepada pengguna dalam bentuk tabel dua dimensi yang terdiri atas baris dan kolom. Kesederhanaan representasi ini sangat kuat karena mudah dipahami oleh manusia sekaligus memiliki landasan matematis yang ketat. Tabel dalam terminologi formal disebut *relasi*, baris disebut *tuple*, dan kolom disebut *atribut*. Kita akan membahas terminologi ini secara mendalam pada bagian berikutnya.

Karakteristik penting lainnya adalah bahwa hubungan antartabel dinyatakan melalui nilai data, khususnya melalui mekanisme kunci (*keys*), bukan melalui pointer fisik seperti pada model hierarkis dan jaringan. Hal ini memberikan fleksibilitas yang luar biasa. Pengguna dapat menggabungkan (*join*) tabel-tabel berdasarkan nilai kolom yang sama tanpa perlu mengetahui bagaimana data disimpan secara fisik. Selain itu, sistem relasional menyediakan bahasa kueri deklaratif yang standar, yaitu SQL

(*Structured Query Language*), yang memungkinkan pengguna menyatakan "apa" data yang diinginkan, bukan "bagaimana" cara mengambilnya.

Untuk memahami mengapa mekanisme kunci begitu fundamental, perlu dibedakan tiga jenis kunci yang berperan dalam menjaga integritas hubungan antartabel. Kunci primer (primary key) adalah atribut atau kombinasi atribut yang secara unik mengidentifikasi setiap baris (tuple) dalam sebuah tabel, sekaligus menjamin tidak ada nilai duplikat maupun nilai *null*. Kunci asing (foreign key) adalah atribut pada satu tabel yang merujuk pada kunci primer tabel lain, sehingga menjadi "jembatan" logis yang menghubungkan kedua entitas. Adapun kunci kandidat (candidate key) adalah himpunan atribut minimal yang memenuhi syarat keunikan dan berpotensi dipilih sebagai kunci primer.

Sebagai ilustrasi konkret, bayangkan tabel *Mahasiswa* dengan kolom *NIM* sebagai kunci primer, dan tabel *Pengambilan_MataKuliah* yang memuat kolom *NIM* sebagai kunci asing. Ketika sistem ingin mengetahui daftar mata kuliah yang diambil seorang mahasiswa, penggabungan dilakukan dengan mencocokkan nilai *NIM* yang identik pada kedua tabel. Perhatikan bahwa yang menautkan kedua baris data bukanlah alamat memori atau pointer yang ditanam saat penyimpanan, melainkan semata-mata *kesamaan nilai*. Konsekuensinya, jika struktur penyimpanan fisik diubah—misalnya data dipindahkan ke disk lain, diindeks ulang, atau dipartisi—hubungan logis antartabel tetap utuh dan tidak perlu didefinisikan ulang.

Fondasi teoretis dari operasi join ini berakar pada aljabar relasional. Operasi *natural join* antara dua relasi

R

R dan

S

S secara formal menghasilkan relasi baru yang berisi kombinasi tuple yang bersesuaian pada atribut bersama, dinotasikan sebagai:

$$R \bowtie S = \{t \mid t[R] \in R \wedge t[S] \in S \wedge t.A = t.A'\}$$

$$R \bowtie S = \{t \mid t[R] \in R \wedge t[S] \in S \wedge t.A = t.A$$

}

Sifat inilah yang membuat model relasional bersifat *set-oriented*: operasi bekerja pada himpunan baris sekaligus, bukan menelusuri satu per satu record melalui rantai pointer sebagaimana pada model jaringan yang menuntut navigasi eksplisit dari programmer.

Keunggulan bahasa kueri deklaratif seperti SQL dapat dipahami melalui pemisahan antara *spesifikasi* dan *eksekusi*. Ketika pengguna menuliskan:

```
SELECT nama FROM Mahasiswa JOIN Pengambilan_MataKuliah ON  
Mahasiswa.NIM = Pengambilan_MataKuliah.NIM WHERE kode_mk = 'IF2210';
```

pengguna sama sekali tidak menentukan urutan pembacaan tabel, penggunaan indeks, ataupun algoritma join yang dipakai. Tugas menerjemahkan permintaan deklaratif menjadi rencana eksekusi yang efisien diserahkan kepada komponen bernama *query optimizer*. Optimizer inilah yang mengevaluasi berbagai kemungkinan strategi pelaksanaan (misalnya *nested-loop join*, *hash join*, atau *merge join*), memperkirakan biaya masing-masing berdasarkan statistik data, lalu memilih rencana dengan biaya estimasi terendah.

Pemisahan ini menghasilkan sejumlah manfaat praktis yang signifikan, antara lain:

- **Kemandirian data fisik (physical data independence):** perubahan pada mekanisme penyimpanan atau penambahan indeks tidak memaksa penulisan ulang kueri aplikasi.
- **Produktivitas pengembang:** logika bisnis dinyatakan secara ringkas dan mendekati bahasa alami, sehingga mengurangi kompleksitas kode navigasional yang rawan kesalahan.
- **Optimasi otomatis dan adaptif:** seiring bertambahnya volume data atau berubahnya distribusi nilai, optimizer dapat menghasilkan rencana eksekusi yang berbeda tanpa intervensi manual dari pengguna.

Dengan demikian, kombinasi antara hubungan berbasis nilai dan bahasa deklaratif bukan sekadar kenyamanan sintaksis, melainkan sebuah perubahan paradigma yang memindahkan beban optimasi dari manusia ke sistem, sekaligus melindungi investasi aplikasi dari perubahan detail implementasi di lapisan penyimpanan.

Sebagai gambaran, sebuah kueri SQL dasar untuk mengambil seluruh data dari sebuah tabel karyawan cukup sederhana seperti berikut:

```
SELECT * FROM employees;
```

Perintah `SELECT *` akan mengembalikan seluruh baris dan kolom dari tabel `employees`. Jika kita hanya menginginkan sebagian data (subset) berdasarkan kriteria tertentu, kita dapat menambahkan kondisi menggunakan klausa `WHERE`, misalnya untuk menampilkan hanya karyawan dengan jabatan tertentu:

```
SELECT first_name, last_name, job_id  
FROM employees
```

```
WHERE job_id = 'SA_REP';
```

Perlu diperhatikan bahwa dalam menuliskan perintah SQL, ejaan yang benar sangatlah penting. Sebagaimana ditekankan dalam materi Oracle, kesalahan penulisan pada kata kunci seperti `SELECT`, nama tabel, atau nama kolom akan menghasilkan pesan kesalahan (*error message*). Ini menuntut ketelitian, karena DBMS akan menolak menjalankan perintah yang tidak sesuai sintaks.

Keunggulan Sistem Basis Data Relasional

Sistem basis data relasional menawarkan berbagai keunggulan yang menjadikannya pilihan utama organisasi. Pertama adalah pengendalian redundansi data, di mana pengulangan data yang tidak perlu dapat diminimalkan melalui teknik normalisasi. Kedua adalah konsistensi dan integritas data, di mana DBMS dapat menegakkan aturan-aturan yang menjamin data selalu valid. Ketiga adalah berbagi data (*data*

sharing), di mana banyak pengguna dan aplikasi dapat mengakses data yang sama secara bersamaan dengan aman.

Keunggulan lainnya mencakup keamanan data melalui mekanisme autentikasi dan otorisasi, independensi data yang memisahkan struktur logis dari implementasi fisik, kemampuan backup dan pemulihan (*recovery*) yang andal, serta penegakan standar data di seluruh organisasi. Semua keunggulan ini secara kolektif menjadikan RDBMS sebagai infrastruktur data yang tangguh, tepercaya, dan skalabel.

Peran DBMS dalam Pengelolaan Data Organisasi

DBMS memainkan peran sentral sebagai penjaga (*custodian*) dan pengelola data organisasi. DBMS menyediakan sejumlah bahasa untuk mengelola basis data.

Berdasarkan buku Gupta dan Mittal, bahasa-bahasa ini mencakup Data Definition Language (DDL) untuk mendefinisikan struktur skema seperti membuat tabel, Data Manipulation Language (DML) untuk memanipulasi data seperti menyisipkan, memperbarui, dan menghapus data, serta bahasa untuk mendefinisikan penyimpanan dan tampilan (*view*). Dalam praktik modern, seluruh bahasa ini terintegrasi dalam SQL.



Ilustrasi konseptual brankas data terpusat sebagai penjaga keamanan basis data, dikelilingi cincin perisai pelindung dan jalur koneksi pengguna yang aman.

Selain itu, DBMS melayani berbagai kategori pengguna. Terdapat pengguna akhir (*end users* atau *naive users*) yang berinteraksi melalui aplikasi tanpa perlu mengetahui SQL, pemrogram aplikasi (*application programmers*) yang membangun aplikasi menggunakan bahasa pemrograman dan SQL, serta administrator basis data (*Database Administrator* atau DBA) yang bertanggung jawab penuh atas pengelolaan, keamanan, kinerja, dan pemeliharaan sistem basis data secara

keseluruhan. Peran DBA sangat strategis karena merekalah yang menjaga agar sumber daya data organisasi tetap aman, tersedia, dan berkinerja tinggi.

Model Data Relasional: Relasi, Atribut, Tuple, dan Domain

Model data relasional adalah jantung dari sistem yang kita pelajari. Model ini dibangun di atas landasan matematis teori himpunan dan logika. Meskipun demikian, penerapannya sangat intuitif karena menyajikan data dalam bentuk tabel yang akrab bagi kita. Mari kita bedah komponen-komponen penyusun model data relasional satu per satu.

Relasi, Tuple, dan Atribut

Dalam model relasional, sebuah relasi (*relation*) adalah representasi formal dari sebuah tabel dua dimensi. Setiap relasi memiliki nama yang unik dalam basis data. Sebuah relasi terdiri atas kumpulan tuple (baris) dan kumpulan atribut (kolom). Sebuah tuple merepresentasikan satu entitas atau satu catatan (record) tunggal, misalnya data seorang mahasiswa tertentu. Sementara itu, sebuah atribut adalah karakteristik atau properti dari entitas yang direpresentasikan oleh relasi, misalnya nama, nomor induk, dan program studi.

Jumlah atribut dalam suatu relasi disebut derajat (*degree*) relasi, sedangkan jumlah tuple dalam relasi disebut kardinalitas (*cardinality*). Perhatikan contoh relasi MAHASISWA berikut sebagai ilustrasi konkret:

Contoh Relasi (Tabel) MAHASISWA dalam Model
Data Relasional

NIM	Nama	Program_Stu di	IPK
2026001	Budi Santoso	Informatika	3.75
2026002	Siti Aminah	Sistem Informasi	3.90
2026003	Andi Wijaya	Informatika	3.45

Pada relasi MAHASISWA di atas, terdapat empat atribut yaitu NIM, Nama, Program_Studi, dan IPK, sehingga derajat relasi ini adalah empat. Terdapat tiga tuple (baris data), sehingga kardinalitas relasi ini adalah tiga. Setiap perpotongan

antara satu tuple dan satu atribut berisi satu nilai tunggal (*atomic value*), yang merupakan salah satu prinsip penting model relasional.

Domain

Domain adalah himpunan semua nilai yang diperbolehkan (valid) untuk suatu atribut. Domain menentukan tipe data dan batasan nilai yang dapat diisikan pada sebuah kolom. Sebagai contoh, domain untuk atribut IPK adalah bilangan pecahan (numerik) dalam rentang 0.00 hingga 4.00. Domain untuk atribut Program_Studi mungkin adalah himpunan nama program studi yang tersedia di fakultas. Konsep domain sangat berkaitan dengan tipe data yang didukung SQL.

Berdasarkan buku *SQL for Beginners*, SQL mendukung berbagai tipe data, di antaranya tipe numerik eksak (*exact numerics*) seperti `INTEGER`, `SMALLINT`, `BIGINT`, `NUMERIC`, dan `DECIMAL`, serta tipe numerik hampiran (*approximate numerics*) seperti `REAL`, `FLOAT`, dan `DOUBLE PRECISION`. Tipe `INTEGER` menyimpan bilangan bulat tanpa titik desimal, sedangkan tipe `NUMERIC` dan `DECIMAL` memungkinkan penyimpanan nilai dengan bagian pecahan yang presisinya dapat didefinisikan. Selain itu terdapat pula tipe string karakter, tipe biner, tipe boolean, dan tipe interval. Pemilihan tipe data yang tepat untuk setiap atribut adalah bagian penting dari perancangan basis data yang baik.

Sebagai ilustrasi konsep presisi dan skala pada tipe `NUMERIC`, jika kita mendefinisikan sebuah kolom dengan presisi 5 dan skala 2, maka kolom tersebut dapat menyimpan nilai hingga 999.99. Presisi menentukan jumlah total digit, sementara skala menentukan jumlah digit di belakang koma. Contohnya, nilai 656.42 memiliki presisi 5 (terdapat 5 digit) dan skala 2 (terdapat 2 desimal).

Kunci (Keys): Primary Key dan Foreign Key

Konsep kunci (*key*) adalah salah satu konsep paling penting dalam model relasional karena kunci inilah yang memungkinkan identifikasi baris secara unik dan pembentukan hubungan antartabel. Terdapat beberapa jenis kunci yang perlu Anda pahami:

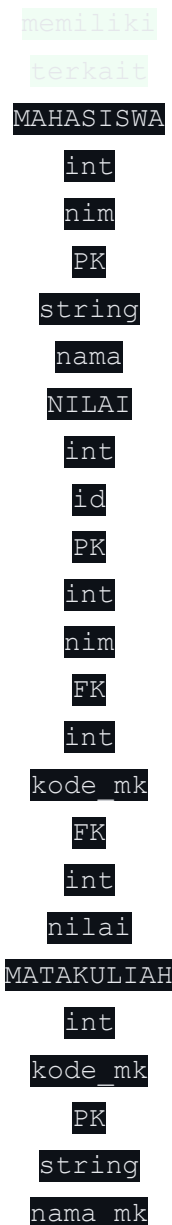


Diagram relasi menunjukkan primary key dan foreign key antar tabel akademik

- Super Key: satu atau kombinasi atribut yang dapat mengidentifikasi setiap tuple secara unik dalam sebuah relasi.
- Candidate Key: super key minimal, artinya tidak ada atribut yang dapat dihilangkan tanpa menghilangkan sifat keunikannya. Sebuah relasi dapat memiliki lebih dari satu candidate key.

- **Primary Key** (kunci utama): satu candidate key yang dipilih untuk menjadi pengidentifikasi utama setiap tuple. Primary key tidak boleh bernilai kosong (NULL) dan harus unik. Pada relasi MAHASISWA, atribut NIM adalah pilihan alami untuk primary key.
- **Foreign Key** (kunci tamu): satu atau lebih atribut dalam sebuah relasi yang merujuk pada primary key relasi lain. Foreign key inilah yang menjadi mekanisme utama untuk membangun hubungan (relationship) antartabel.

Sebagai ilustrasi hubungan antartabel melalui foreign key, perhatikan bagaimana relasi NILAI dapat merujuk pada relasi MAHASISWA dan MATA_KULIAH:

```

memperoleh
dinilai_pada
MAHASISWA
string
NIM
PK
string
Nama
string
Program_Studi
float
IPK
NILAI
string
NIM
FK
string
Kode_MK
FK
float
Nilai_Angka
MATA_KULIAH
string
Kode_MK
PK
string
Nama_MK
int
SKS

```

Pada diagram di atas, tabel NILAI memiliki dua foreign key, yaitu NIM yang merujuk pada MAHASISWA dan Kode_MK yang merujuk pada MATA_KULIAH. Dengan demikian, kita dapat menelusuri nilai yang diperoleh setiap mahasiswa pada setiap mata kuliah tanpa mengulang seluruh data mahasiswa maupun mata kuliah dalam tabel NILAI. Inilah kekuatan model relasional dalam mengurangi redundansi sekaligus menjaga keterhubungan data.

Integritas Relasional

Untuk menjamin kualitas dan konsistensi data, model relasional menegakkan sejumlah aturan integritas (*integrity constraints*). Terdapat dua aturan integritas fundamental. Pertama adalah integritas entitas (*entity integrity*), yang menyatakan bahwa nilai primary key tidak boleh kosong (NULL) dan harus unik. Aturan ini memastikan setiap baris dapat diidentifikasi secara pasti. Kedua adalah integritas referensial (*referential integrity*), yang menyatakan bahwa nilai foreign key harus cocok dengan salah satu nilai primary key pada tabel yang dirujuk, atau bernilai NULL. Aturan ini mencegah adanya "referensi menggantung" (*dangling reference*), misalnya mencatat nilai untuk mahasiswa yang tidak ada dalam tabel MAHASISWA.

Selain kedua aturan tersebut, terdapat pula integritas domain yang memastikan setiap nilai atribut sesuai dengan domain yang telah ditentukan, serta berbagai integritas yang ditentukan pengguna (*user-defined integrity*) yang mencerminkan aturan bisnis spesifik organisasi, misalnya IPK tidak boleh melebihi 4.00.

Aturan-aturan integritas ini ditegakkan secara otomatis oleh DBMS, sehingga membebaskan pemrogram dari keharusan memeriksa validitas data secara manual pada setiap aplikasi.

Arsitektur DBMS: Arsitektur Tiga Skema dan Independensi Data

Salah satu kontribusi terpenting dari teori basis data adalah gagasan pemisahan pandangan (view) terhadap data pada tingkat yang berbeda. Gagasan ini dituangkan dalam arsitektur tiga skema (*three-schema architecture*), yang juga dikenal sebagai arsitektur ANSI/SPARC. Arsitektur ini dirancang untuk mencapai tujuan utama, yaitu memisahkan aplikasi pengguna dari basis data fisik, sehingga menghasilkan independensi data.

Tiga Tingkat Arsitektur

Berdasarkan buku Gupta dan Mittal, arsitektur tiga skema membagi deskripsi basis data ke dalam tiga tingkat atau pandangan (level/views) sebagai berikut:

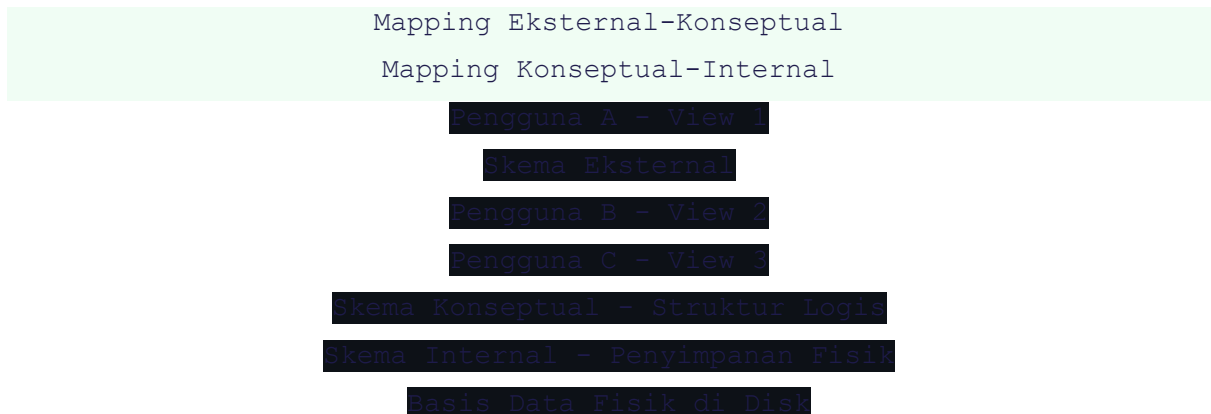


Arsitektur tiga skema DBMS: tingkat eksternal, konseptual, dan internal

- **Tingkat Internal (Internal/Physical Level):** tingkat terendah yang menjelaskan bagaimana data disimpan secara fisik di media penyimpanan. Skema internal mendeskripsikan struktur penyimpanan, metode akses, indeks, kompresi data, dan detail teknis lainnya. Tingkat ini berorientasi pada efisiensi penyimpanan dan kinerja.
- **Tingkat Konseptual (Conceptual/Logical Level):** tingkat menengah yang menjelaskan struktur logis keseluruhan basis data untuk seluruh komunitas pengguna. Skema konseptual mendeskripsikan entitas, atribut, hubungan, dan batasan-batasan (constraints), tetapi menyembunyikan detail penyimpanan fisik. Inilah tingkat yang menjadi fokus perancang basis data.

- **Tingkat Eksternal (External/View Level):** tingkat tertinggi yang menjelaskan bagian-bagian basis data yang relevan bagi kelompok pengguna tertentu. Setiap skema eksternal (subschema) menyajikan tampilan (view) yang disesuaikan dengan kebutuhan pengguna, sekaligus menyembunyikan bagian data yang tidak relevan atau tidak boleh diakses pengguna tersebut.

Hubungan antartingkat dalam arsitektur tiga skema dapat digambarkan sebagai berikut:



Pemetaan (Mapping)

Agar ketiga tingkat tersebut dapat bekerja bersama, DBMS melakukan dua jenis pemetaan (*mapping*). Pertama adalah pemetaan eksternal-konseptual yang menghubungkan setiap skema eksternal dengan skema konseptual, memungkinkan DBMS menerjemahkan permintaan pengguna pada tingkat view menjadi operasi pada tingkat logis. Kedua adalah pemetaan konseptual-internal yang menghubungkan skema konseptual dengan skema internal, memungkinkan DBMS menerjemahkan struktur logis menjadi struktur penyimpanan fisik.

Pemetaan-pemetaan inilah yang menjadi kunci tercapainya independensi data.

Independensi Data

Independensi data (*data independence*) adalah kemampuan untuk mengubah skema pada satu tingkat tanpa harus mengubah skema pada tingkat yang lebih tinggi. Terdapat dua jenis independensi data. Pertama adalah independensi data

logis (*logical data independence*), yaitu kemampuan mengubah skema konseptual (misalnya menambahkan atribut atau tabel baru) tanpa harus mengubah skema eksternal atau program aplikasi yang ada. Kedua adalah independensi data fisik (*physical data independence*), yaitu kemampuan mengubah skema internal (misalnya mengubah metode penyimpanan atau menambah indeks) tanpa harus mengubah skema konseptual.

Independensi data merupakan keunggulan besar dibandingkan sistem berkas tradisional yang memiliki masalah ketergantungan program terhadap data. Dengan independensi data, seorang administrator dapat mengoptimalkan penyimpanan fisik demi peningkatan kinerja tanpa mengganggu aplikasi yang berjalan, dan seorang perancang dapat mengembangkan struktur logis basis data seiring berkembangnya kebutuhan organisasi tanpa merombak seluruh sistem. Inilah fleksibilitas yang membuat sistem basis data modern begitu tangguh dan mudah dipelihara.

Komponen Utama DBMS

Sebuah DBMS terdiri atas beberapa komponen utama yang bekerja secara terkoordinasi. Komponen-komponen ini mencakup pengolah kueri (*query processor*) yang mengurai, mengoptimalkan, dan mengeksekusi kueri; pengelola penyimpanan (*storage manager*) yang mengatur penyimpanan dan pengambilan data di disk; pengelola transaksi (*transaction manager*) yang menjamin sifat ACID (Atomicity, Consistency, Isolation, Durability) dari setiap transaksi; pengelola buffer (*buffer manager*) yang mengelola data di memori; serta pengelola pemulihan (*recovery manager*) yang memastikan basis data dapat dipulihkan setelah kegagalan. Semua komponen ini bekerja di balik layar untuk memberikan layanan pengelolaan data yang andal.

Arsitektur Oracle dan PostgreSQL

Setelah memahami konsep arsitektur DBMS secara umum, mari kita lihat bagaimana konsep-konsep tersebut diwujudkan dalam dua DBMS relasional terkemuka yang menjadi fokus mata kuliah ini, yaitu Oracle dan PostgreSQL. Kedua sistem ini sama-sama merupakan RDBMS yang matang dan tangguh, tetapi memiliki perbedaan filosofi dan implementasi arsitektural yang menarik untuk dibandingkan. Perlu dicatat bahwa pembahasan berikut bersifat gambaran umum konseptual pada level C2 (memahami), bukan konfigurasi teknis mendalam.

Gambaran Umum Arsitektur Oracle

Secara konseptual, arsitektur Oracle Database terdiri atas dua komponen besar, yaitu instance dan database. *Instance* adalah kombinasi antara struktur memori dan sekumpulan proses latar belakang (*background processes*) yang berjalan di server pada saat basis data aktif. Struktur memori utama pada instance Oracle disebut System Global Area (SGA), yaitu area memori bersama yang digunakan oleh semua proses untuk menyimpan data yang di-cache, informasi kamus data, serta kode SQL yang telah dikompilasi. Selain itu terdapat Program Global Area (PGA) yang merupakan area memori privat untuk setiap proses server.



Gambaran umum arsitektur Oracle: instance memori dan penyimpanan basis data

Sementara itu, komponen *database* pada Oracle merujuk pada kumpulan berkas fisik yang menyimpan data secara permanen di disk. Berkas-berkas ini meliputi data files yang menyimpan data aktual, control files yang menyimpan metadata tentang struktur fisik basis data, dan redo log files yang mencatat setiap perubahan untuk

keperluan pemulihan. Oracle juga menjalankan sejumlah proses latar belakang penting seperti Database Writer, Log Writer, dan Checkpoint yang masing-masing memiliki tugas spesifik dalam menjaga konsistensi dan ketahanan data. Dalam mata kuliah ini, kita juga akan menggunakan Oracle APEX (Application Express), sebuah platform pengembangan aplikasi berbasis web yang memungkinkan banyak pengembang membangun dan mengakses aplikasi seolah-olah berjalan di basis data terpisah. Oracle APEX memiliki dua komponen utama, yaitu SQL Workshop untuk mempelajari dan menjalankan SQL, serta Application Builder untuk merancang aplikasi.

Gambaran Umum Arsitektur PostgreSQL

PostgreSQL adalah RDBMS bersumber terbuka (open source) yang sangat dihormati karena kepatuhannya terhadap standar SQL dan keandalannya. Arsitektur PostgreSQL menggunakan model proses-per-koneksi (*process-per-connection*), di mana terdapat satu proses induk bernama *postmaster* yang menerima koneksi masuk dan menciptakan proses server terpisah (backend process) untuk melayani setiap klien. Model ini berbeda dengan beberapa DBMS lain yang menggunakan model berbasis thread.

Dari sisi memori, PostgreSQL menggunakan area memori bersama yang disebut *shared buffers* untuk menyimpan cache halaman data, mirip dengan fungsi SGA pada Oracle meskipun implementasinya berbeda. PostgreSQL juga memiliki berbagai proses latar belakang seperti *background writer*, *WAL writer* (Write-Ahead Logging), *autovacuum*, dan *checkpointer*. Mekanisme WAL pada PostgreSQL setara secara konsep dengan redo log pada Oracle, yaitu mencatat perubahan sebelum diterapkan ke data files demi menjamin ketahanan dan pemulihan. Dari sisi penyimpanan, PostgreSQL menyimpan data dalam berkas-berkas yang terorganisasi dalam direktori data (*data directory*).

Kedua DBMS ini mengimplementasikan konsep arsitektur tiga skema dan independensi data yang telah kita bahas. Keduanya memisahkan tampilan logis (tabel, view) dari penyimpanan fisik (data files), sehingga administrator dapat mengoptimalkan penyimpanan tanpa mengganggu aplikasi. Perbedaan detail arsitektural antara Oracle dan PostgreSQL akan menjadi bahan diskusi menarik sepanjang perkuliahan, terutama ketika kita membahas topik optimisasi performa dan manajemen transaksi pada pertemuan-pertemuan berikutnya.

Studi Kasus: Merancang Basis Data Akademik Sederhana

Untuk mengonsolidasikan seluruh konsep yang telah kita pelajari, mari kita telaah sebuah studi kasus nyata yang relevan dengan lingkungan Anda sebagai mahasiswa: perancangan basis data akademik sederhana untuk sebuah program studi. Studi kasus ini akan menunjukkan bagaimana konsep data, informasi, relasi, atribut, kunci, dan integritas relasional diterapkan secara terpadu.



Ilustrasi mahasiswa Indonesia berdiskusi dan bekerja di laboratorium komputer dengan skema basis data ditampilkan pada layar monitor.

Bayangkan sebuah program studi ingin mengelola data akademiknya secara digital. Sebelum adanya sistem basis data, mereka mungkin menggunakan berkas lembar sebar (spreadsheet) terpisah: satu berkas untuk data mahasiswa, satu berkas untuk data mata kuliah, dan satu berkas lagi untuk nilai. Pendekatan berbasis berkas ini menimbulkan masalah yang telah kita bahas: ketika seorang mahasiswa mengubah program studinya, staf harus memperbarui data tersebut di banyak tempat, yang berisiko menimbulkan inkonsistensi. Selain itu, sulit menjawab pertanyaan kompleks

seperti "berapa rata-rata nilai mahasiswa Informatika pada mata kuliah tertentu?" tanpa kerja manual yang melelahkan.

Dengan pendekatan sistem basis data relasional, kita merancang tiga relasi utama. Relasi MAHASISWA dengan primary key NIM, relasi MATA_KULIAH dengan primary key Kode_MK, dan relasi NILAI yang menghubungkan keduanya melalui dua foreign key. Perancangan ini menerapkan prinsip pengendalian redundansi: data lengkap mahasiswa hanya disimpan sekali di tabel MAHASISWA, data lengkap mata kuliah hanya disimpan sekali di tabel MATA_KULIAH, dan tabel NILAI cukup menyimpan referensi berupa NIM dan Kode_MK beserta nilainya.

Mari kita lihat bagaimana proses pembuatan struktur tabel ini dilakukan menggunakan DDL dalam SQL. Perlu diingat bahwa contoh berikut merupakan ilustrasi konseptual sederhana untuk memperkuat pemahaman, bukan implementasi lengkap:

```
CREATE TABLE mahasiswa (  
  nim          VARCHAR(10) PRIMARY KEY,  
  nama         VARCHAR(50) NOT NULL,  
  program_studi VARCHAR(30),  
  ipk          NUMERIC(3,2)  
);
```

```
CREATE TABLE mata_kuliah (  
  kode_mk      VARCHAR(8) PRIMARY KEY,  
  nama_mk      VARCHAR(50) NOT NULL,  
  sks          INTEGER  
);
```

```
CREATE TABLE nilai (  
  nim          VARCHAR(10),  
  kode_mk      VARCHAR(8),  
  nilai_angka  NUMERIC(4,2),  
  PRIMARY KEY (nim, kode_mk),  
  FOREIGN KEY (nim) REFERENCES mahasiswa(nim),  
  FOREIGN KEY (kode_mk) REFERENCES mata_kuliah(kode_mk)  
);
```

Perhatikan bagaimana konsep-konsep teoretis diwujudkan dalam kode di atas.

Klausula `PRIMARY KEY` menegaskan integritas entitas, klausula `NOT NULL` memastikan atribut penting tidak kosong, klausula `FOREIGN KEY ... REFERENCES` menegaskan integritas referensial, dan penentuan tipe data seperti `NUMERIC(3,2)` menegaskan integritas domain. Tabel NILAI menggunakan primary key gabungan (*composite key*) dari NIM dan Kode_MK, karena satu mahasiswa dapat mengambil banyak mata kuliah dan satu mata kuliah dapat diambil banyak mahasiswa (hubungan banyak-ke-banyak).

Setelah struktur dan data terisi, kita dapat dengan mudah menjawab pertanyaan-pertanyaan analitis yang tadinya sulit. Sebagai contoh, untuk menampilkan nama mahasiswa beserta nilai mata kuliah yang mereka ambil, kita menggabungkan tabel dengan operasi JOIN. Kemampuan ini menunjukkan keunggulan integrasi data pada sistem relasional. Berkat independensi data, jika suatu saat administrator menambahkan indeks pada kolom NIM untuk mempercepat pencarian, aplikasi yang menggunakan kueri di atas tidak perlu diubah sama sekali. Studi kasus ini memperlihatkan bagaimana teori yang kita pelajari secara langsung menyelesaikan masalah nyata dalam pengelolaan data organisasi, sekaligus menjadi bekal untuk topik desain skema dan normalisasi pada pertemuan mendatang.

Tabel Perbandingan dan Ringkasan Konsep

Untuk membantu Anda mengonsolidasikan pemahaman, berikut disajikan beberapa tabel perbandingan yang merangkum konsep-konsep kunci yang telah dibahas.

Tabel-tabel ini dapat Anda gunakan sebagai bahan tinjauan cepat menjelang quiz maupun ujian.

Perbandingan Sistem Berkas Tradisional dengan
Sistem Basis Data

Aspek	Sistem Berkas Tradisional	Sistem Basis Data (DBMS)
Redundansi Data	Tinggi, data sering diduplikasi	Terkontrol dan diminimalkan
Konsistensi Data	Rentan inkonsistensi	Terjaga melalui aturan integritas
Integritas Data	Sulit ditegakkan	Ditegakkan otomatis oleh DBMS
Berbagi Data	Terbatas dan sulit	Mendukung banyak pengguna bersamaan

Keamanan	Lemah	Autentikasi dan otorisasi terpusat
Independensi Data	Program bergantung pada data	Independensi logis dan fisik
Pemulihan (Recovery)	Sulit dan manual	Mekanisme backup dan recovery andal

Tabel berikut merangkum terminologi model relasional beserta padanan informalnya, yang sering menjadi sumber kebingungan bagi pemula. Menguasai terminologi ini akan sangat membantu Anda memahami literatur akademik maupun dokumentasi teknis.

Terminologi Formal Model Relasional dan Padanan Informalnya

Istilah Formal	Padanan Informal	Penjelasan Singkat
----------------	------------------	--------------------

Relasi (Relation)	Tabel	Struktur dua dimensi berisi data
-------------------	-------	----------------------------------

Tuple	Baris / Record	Satu catatan entitas tunggal
-------	----------------	------------------------------

Atribut (Attribute)	Kolom / Field	Karakteristik atau properti entitas
---------------------	---------------	-------------------------------------

Domain	Tipe & rentang nilai	Himpunan nilai valid untuk atribut
--------	----------------------	------------------------------------

Derajat (Degree)	Jumlah kolom	Banyaknya atribut dalam relasi
------------------	--------------	--------------------------------

Kardinalitas (Cardinality)	Jumlah baris	Banyaknya tuple dalam relasi
-------------------------------	--------------	------------------------------

Tabel terakhir menyajikan perbandingan gambaran umum arsitektur Oracle dan PostgreSQL. Perlu diingat bahwa perbandingan ini bersifat konseptual dan umum; detail teknis lebih lanjut akan dibahas pada pertemuan yang relevan.

Perbandingan Gambaran Umum Arsitektur
Oracle dan PostgreSQL

Aspek	Oracle	PostgreSQL
Jenis Lisensi	Komersial (berbayar)	Open source (gratis)
Model Proses	Instance dengan proses latar belakang	Proses-per-koneksi (postmaster)
Memori Bersama	System Global Area (SGA)	Shared Buffers
Mekanisme Log Pemulihan	Redo Log Files	Write-Ahead Logging (WAL)

Platform	Oracle APEX	Berbagai tool (mis. pgAdmin)
Pengembangan		

Ketiga tabel di atas memberikan gambaran menyeluruh yang menghubungkan konsep teoretis dengan implementasi praktis. Perhatikan bahwa meskipun Oracle dan PostgreSQL berbeda dalam detail implementasi, keduanya menerapkan prinsip-prinsip fundamental yang sama: model relasional, arsitektur tiga skema, independensi data, dan penegakan integritas. Ini menegaskan bahwa penguasaan konsep dasar jauh lebih penting daripada sekadar menghafal fitur spesifik satu produk, karena konsep dasar bersifat universal dan dapat diterapkan pada DBMS mana pun.

Latihan dan Pertanyaan Diskusi

Untuk mengukur dan memperdalam pemahaman Anda terhadap materi pertemuan ini, kerjakanlah latihan dan diskusikan pertanyaan-pertanyaan berikut. Latihan ini dirancang sesuai dengan level kognitif C2 (Memahami), sehingga fokusnya adalah menjelaskan, menginterpretasikan, membandingkan, dan memberikan contoh, bukan sekadar menghafal.

Pertanyaan Pemahaman Konsep

1. Jelaskan dengan bahasa Anda sendiri perbedaan antara data dan informasi. Berikan satu contoh konkret dari lingkungan kampus yang menunjukkan bagaimana data ditransformasikan menjadi informasi.
2. Uraikan minimal empat kelemahan sistem berkas tradisional dan jelaskan bagaimana sistem basis data mengatasi masing-masing kelemahan tersebut.

3. Apa yang dimaksud dengan metadata dan kamus data? Berikan contoh metadata untuk sebuah kolom yang menyimpan tanggal lahir mahasiswa.
4. Jelaskan perbedaan antara perangkat lunak sistem dan perangkat lunak aplikasi, serta tentukan pada kategori mana DBMS berada. Sertakan alasan Anda.
5. Mengapa Dr. E. F. Codd dianggap sebagai tokoh paling berpengaruh dalam sejarah basis data relasional? Jelaskan kontribusi utamanya.

Pertanyaan Analisis Model Relasional

1. Diberikan sebuah relasi PEGAWAI dengan atribut (ID_Pegawai, Nama, Departemen, Gaji). Tentukan kandidat untuk primary key dan jelaskan alasannya. Berapakah derajat relasi ini?
2. Jelaskan perbedaan antara primary key dan foreign key, serta berikan contoh bagaimana keduanya digunakan untuk menghubungkan dua tabel.
3. Apa yang dimaksud dengan integritas entitas dan integritas referensial? Berikan contoh pelanggaran terhadap masing-masing aturan tersebut.
4. Sebuah kolom didefinisikan dengan tipe `NUMERIC (6, 2)`. Jelaskan berapa nilai maksimum yang dapat disimpan pada kolom tersebut dan mengapa.
5. Mengapa tabel penghubung (seperti tabel NILAI pada studi kasus) sering menggunakan primary key gabungan? Jelaskan dengan konsep hubungan banyak-ke-banyak.

Pertanyaan Diskusi Arsitektur

1. Gambarkan dan jelaskan arsitektur tiga skema. Mengapa pemisahan tingkat internal, konseptual, dan eksternal dianggap penting?
2. Jelaskan perbedaan antara independensi data logis dan independensi data fisik. Berikan satu skenario praktis untuk masing-masing.
3. Bandingkan konsep instance dan database pada arsitektur Oracle. Apa fungsi utama dari System Global Area (SGA)?
4. Apa persamaan konseptual antara redo log pada Oracle dan Write-Ahead Logging (WAL) pada PostgreSQL? Mengapa mekanisme semacam ini diperlukan?
5. Diskusikan dalam kelompok: dalam situasi seperti apa sebuah organisasi mungkin lebih memilih PostgreSQL daripada Oracle, dan sebaliknya? Pertimbangkan aspek biaya, dukungan, dan kebutuhan fitur.

Latihan Praktik SQL Sederhana

Sebagai latihan awal untuk membiasakan diri dengan sintaks SQL, tuliskan perintah SQL untuk kebutuhan berikut, dengan asumsi telah tersedia tabel `employees` dengan kolom `first_name`, `last_name`, dan `job_id`:

1. Tampilkan seluruh kolom dan seluruh baris dari tabel `employees`.
2. Tampilkan hanya kolom nama depan dan nama belakang untuk seluruh karyawan.
3. Tampilkan nama depan, nama belakang, dan `job_id` untuk karyawan yang memiliki jabatan dengan kode `'IT_PROG'`.
4. Identifikasi kesalahan yang mungkin terjadi jika Anda salah mengetik nama tabel atau nama kolom, dan jelaskan cara memperbaikinya.

Kerjakan latihan-latihan ini secara mandiri terlebih dahulu, kemudian diskusikan jawaban Anda dengan teman sekelas atau pada forum diskusi. Aktivitas diskusi sangat dianjurkan karena sesuai dengan metode pembelajaran pertemuan ini, sekaligus membantu Anda menemukan sudut pandang baru dan mengklarifikasi miskonsepsi. Ingatlah bahwa pada level C2, kemampuan menjelaskan alasan di balik jawaban lebih dihargai daripada sekadar jawaban yang benar tanpa penjelasan.

Rangkuman dan Kesimpulan

Pertemuan pertama ini telah membawa kita menelusuri fondasi paling mendasar dari sistem basis data relasional. Kita memulai dengan membedakan konsep-konsep fundamental: data sebagai fakta mentah, informasi sebagai data yang telah diproses dan bermakna, basis data sebagai kumpulan data yang saling berhubungan dan terorganisasi, serta sistem basis data sebagai lingkungan menyeluruh yang mencakup perangkat keras, perangkat lunak DBMS, data, prosedur, dan pengguna. Kita juga memahami peran metadata dan kamus data dalam mendeskripsikan struktur data itu sendiri.

Selanjutnya, kita menelaah mengapa sistem basis data lahir sebagai jawaban atas berbagai kelemahan sistem berkas tradisional, seperti redundansi data, inkonsistensi, isolasi data, lemahnya integritas, kesulitan akses bersamaan, serta ketergantungan program terhadap data. Kita menempatkan hal ini dalam konteks historis, mulai dari model hierarkis dan jaringan, hingga momen revolusioner ketika Dr. E. F. Codd memperkenalkan model relasional pada tahun 1970 yang menjadi cikal bakal seluruh RDBMS modern termasuk Oracle dan PostgreSQL.

Kita kemudian membedah model data relasional secara mendalam. Kita mempelajari bahwa data disajikan dalam bentuk relasi (tabel) yang terdiri atas tuple (baris) dan atribut (kolom), dengan setiap atribut memiliki domain yang menentukan nilai-nilai valid. Kita memahami hierarki kunci mulai dari super key, candidate key, hingga primary key yang mengidentifikasi baris secara unik, serta foreign key yang menghubungkan antartabel. Kita juga menegaskan pentingnya aturan-aturan integritas relasional, khususnya integritas entitas dan integritas referensial, yang dijaga secara otomatis oleh DBMS untuk menjamin kualitas dan konsistensi data.

Pada aspek arsitektur, kita mempelajari arsitektur tiga skema (ANSI/SPARC) yang memisahkan tingkat internal (fisik), konseptual (logis), dan eksternal (view).

Pemisahan ini menghasilkan independensi data, baik independensi logis maupun fisik, yang merupakan keunggulan besar dibandingkan sistem berkas. Kita juga mengenal komponen-komponen utama DBMS seperti pengolah kueri, pengelola penyimpanan, pengelola transaksi, dan pengelola pemulihan. Semua konsep ini kemudian kita hubungkan dengan implementasi nyata pada arsitektur Oracle (dengan konsep instance, SGA, redo log, dan platform Oracle APEX) dan PostgreSQL (dengan model proses-per-koneksi, shared buffers, dan WAL).

Melalui studi kasus perancangan basis data akademik sederhana, kita menyaksikan bagaimana seluruh konsep teoretis tersebut bekerja secara terpadu dalam menyelesaikan masalah nyata: mengurangi redundansi, menjaga konsistensi, menegakkan integritas, dan memungkinkan analisis data yang fleksibel. Kita bahkan

telah mengintip sintaks dasar SQL, baik untuk mendefinisikan struktur (DDL) maupun untuk mengambil data (SELECT), sebagai jembatan menuju pembahasan SQL yang lebih komprehensif pada pertemuan mendatang.

Sebagai penutup, penting untuk ditegaskan bahwa pemahaman konseptual yang Anda peroleh pada pertemuan ini adalah bekal yang tak ternilai. Seluruh topik lanjutan dalam mata kuliah ini, mulai dari pemodelan Entity-Relationship, desain skema, normalisasi, penulisan kueri SQL yang kompleks, manajemen transaksi, keamanan, hingga optimisasi performa, semuanya dibangun di atas fondasi yang telah kita letakkan hari ini. Konsep-konsep dasar bersifat universal dan tidak lekang oleh waktu; meskipun teknologi dan produk DBMS terus berkembang, prinsip model relasional dan arsitektur DBMS tetap menjadi landasan. Oleh karena itu, pastikan Anda benar-benar menguasai materi ini sebelum melangkah ke pertemuan berikutnya. Manfaatkan latihan dan pertanyaan diskusi yang telah disediakan, dan jangan ragu untuk kembali menelaah bagian-bagian yang masih terasa sulit. Dengan pemahaman yang kokoh, Anda akan mampu menganalisis kebutuhan, merancang solusi basis data yang efektif, dan pada akhirnya menjadi profesional teknologi informasi yang kompeten dan siap menghadapi tantangan pengelolaan data di dunia nyata.